SISTEMAS EMBEBIDOS

Universidad Politécnica del

Bicentenario.

Sistema de Control PID de Temperatura con ESP32, Servomotor y Dimmer de Potencia

Ingeniería Mecatrónica 701

Edgar Uriel Rodríguez Moreno *24033660*

Jorge Zamorano Chávez *24033247*

Axel Gabriel Domínguez Macías *24032952*

Braulio Vázquez Morado *24032975*

Daisy Jackeline Hernández Ramos *24033069*

Silao, Guanajuato a 12 de Noviembre de 2026.

Introducción

El controlador Proporcional, Integral y Derivativo representa la estrategia de regulación automática más versátil y extendida dentro de la ingeniería de control y la automatización industrial. A diferencia del control todo-nada, que genera oscilaciones permanentes alrededor del punto de consigna, y del control puramente proporcional, que presenta un error residual en estado estable ante perturbaciones térmicas o pérdidas de calor hacia el entorno, el algoritmo PID combina tres acciones matemáticas complementarias para gobernar con alta precisión procesos dinámicos no lineales o con inercia térmica considerable. La acción proporcional responde de forma inmediata a la magnitud del error actual; la acción integral acumula el error histórico a lo largo del tiempo para garantizar que la variable controlada alcance exactamente la temperatura deseada sin desviaciones permanentes; y la acción derivativa evalúa la velocidad de cambio de la temperatura para anticipar tendencias, amortiguar el sistema y reducir el sobreimpulso antes de alcanzar la referencia.

El presente reporte documenta el diseño, la implementación discreta, la calibración electromecánica y la validación experimental de un sistema de control PID de temperatura en lazo cerrado gobernado por un microcontrolador ESP32. El prototipo emplea un sensor digital DS18B20 para adquirir la temperatura interna de una cámara térmica aislada, un teclado matricial de 4×4 para sintonizar en tiempo real la temperatura de referencia y las tres ganancias de control, y una pantalla LCD de 16×2 con interfaz I2C que alterna dinámicamente entre la visualización de las variables de proceso y los parámetros de diagnóstico. Para modular la energía entregada al elemento calefactor de corriente alterna sin comprometer el aislamiento galvánico de la etapa digital, la salida porcentual calculada por el algoritmo PID discreto el cual incorpora protección *anti-windup* y cálculo derivativo sobre la medición se transforma en una consigna angular ejecutada por un servomotor SG90 acoplado mecánicamente a la perilla de un dimmer regulador de potencia.

Marco Teórico

El funcionamiento integral del lazo de control PID se sustenta en la interacción coordinada de diversos componentes electrónicos, transductores, actuadores y algoritmos discretos, cuyas características técnicas y compatibilidad de programación se describen a continuación.

El *microcontrolador ESP32 DevKit V1* constituye la unidad central de procesamiento, adquisición y cálculo numérico del sistema. Este System-on-Chip de 32 bits desarrollado por Espressif Systems integra un procesador Xtensa LX6 de doble núcleo capaz de operar hasta a 240 MHz, unidad de punto flotante por hardware ideal para ejecutar ecuaciones diferenciales discretizadas en tiempo real, memoria SRAM interna y periféricos de comunicación serie como I2C, SPI y UART, además de controladores dedicados de modulación por ancho de pulso LEDC de alta precisión. En esta práctica, el ESP32 ejecuta cada segundo el lazo cerrado PID, administra las interrupciones y lecturas del teclado matricial, refresca el display LCD y genera la señal PWM de posicionamiento del servomotor. Respecto a su compatibilidad de programación, soporta C y C++ a través del framework Arduino y el entorno oficial ESP-IDF en PlatformIO o Arduino IDE, así como MicroPython, CircuitPython, Lua, Rust y JavaScript.

El *sensor digital de temperatura DS18B20* opera como el elemento primario de retroalimentación térmica del sistema. Fabricado por Maxim Integrated, actualmente Analog Devices, mide temperaturas en el rango de -55 °C a +125 °C con una precisión de ±0.5 °C y resolución configurable entre 9 y 12 bits, empleando en este proyecto una resolución de 10 bits para lograr un equilibrio óptimo entre rapidez de conversión y sensibilidad ante variaciones pequeñas de temperatura. Al transmitir los datos en formato digital mediante el bus propietario 1-Wire asistido por una resistencia pull-up de 4.7 kΩ, resulta altamente inmune al ruido electromagnético provocado por el recorte de fase del dimmer de corriente alterna. En cuanto a lenguajes de programación, es compatible con C y C++ mediante las bibliotecas OneWire y DallasTemperature en Arduino y PlatformIO, MicroPython con los

módulos ds18x20 y onewire, Python en plataformas Linux embebidas, C estándar en microcontroladores AVR o PIC, y Rust.

La *pantalla LCD alfanumérica de 16×2 con adaptador I2C* funciona como la interfaz visual de supervisión local. Basada en el controlador HD44780 acoplado a un expansor de entradas y salidas PCF8574, permite desplegar dos filas de dieciséis caracteres utilizando únicamente las líneas SDA y SCL del bus I2C, reduciendo el consumo de pines GPIO e incorporando ajuste de contraste y retroiluminación. En este controlador PID alterna automáticamente cada cuatro segundos o de forma manual con la tecla entre una vista operativa con temperatura medida, setpoint, error y porcentaje de salida, y una vista de diagnóstico con las ganancias proporcional, integral y derivativa junto al ángulo exacto del servomotor. Este periférico es compatible con C y C++ mediante librerías como LiquidCrystal_I2C en Arduino y PlatformIO o controladores I2C nativos en ESP-IDF, además de MicroPython, Python, CircuitPython y ensamblador.

El *teclado matricial de membrana 4×4* proporciona la interfaz de sintonización en línea del controlador. Compuesto por dieciséis interruptores dispuestos en una matriz de cuatro filas por cuatro columnas, permite al microcontrolador identificar cualquier pulsación mediante barrido secuencial digital. En este sistema, las teclas A, B, C y D seleccionan respectivamente la edición de la temperatura de referencia, la ganancia proporcional, la ganancia integral y la ganancia derivativa; los dígitos numéricos y el asterisco * permiten ingresar valores con punto decimal, y la tecla # valida y aplica el nuevo parámetro en tiempo real sin detener el control térmico. Su lectura es compatible con C y C++ mediante la librería Keypad en PlatformIO y Arduino o rutinas matriciales en ESP-IDF, MicroPython, CircuitPython, Python y Rust.

El *servomotor micro SG90* actúa como el elemento transductor electromecánico que convierte la señal de control porcentual en desplazamiento angular físico. Integra un motor de corriente continua, un tren de engranajes reductores, un potenciómetro interno de retroalimentación de posición y un circuito de control que responde a

pulsos PWM de 50 Hz con anchos entre 500 y 2400 microsegundos. Gracias a su torque de retención, gira y sostiene con precisión la perilla del dimmer dentro del rango mecánico calibrado. Respecto a su compatibilidad de software, es gobernado mediante C y C++ utilizando bibliotecas como ESP32Servo o Servo en PlatformIO y Arduino, así como con MicroPython, CircuitPython, Python en Raspberry Pi, Rust y lenguaje ensamblador.

El *módulo dimmer regulador de potencia AC* constituye la etapa final de potencia encargada de modular el flujo energético hacia el calefactor. Conformado por un TRIAC, un DIAC, una red RC de disparo y un potenciómetro rotativo, controla el ángulo de conducción de cada semiciclo de la tensión alterna conforme gira su perilla. Al estar acoplado mecánicamente al servomotor, permite variar gradualmente el voltaje eficaz aplicado al calefactor manteniendo una separación física completa respecto a la electrónica de control. Por su accionamiento puramente mecánico gobernado por el servomotor, es compatible con cualquier lenguaje de programación capaz de generar señales PWM en el microcontrolador, incluyendo C, C++, MicroPython, Python y Rust.

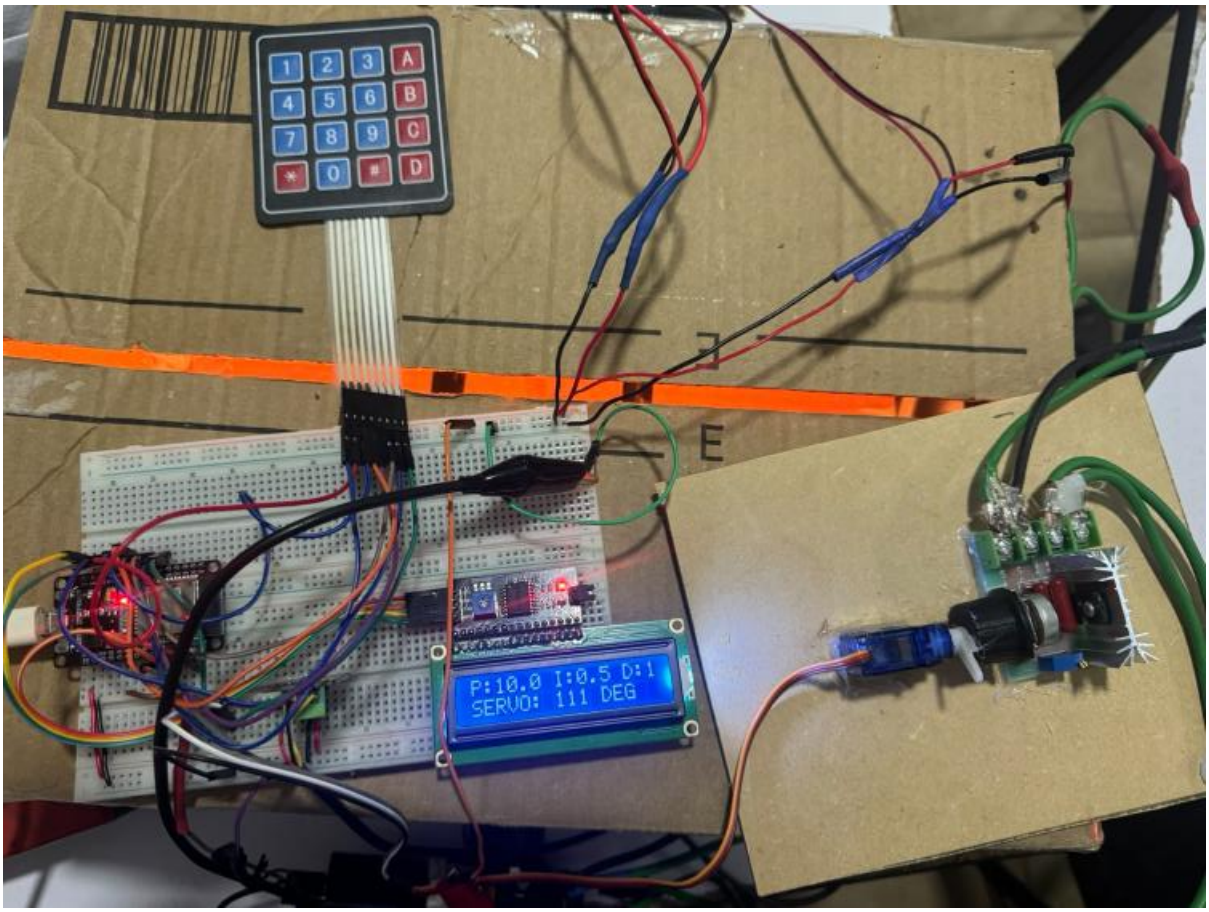
El *elemento calefactor y la cámara térmica aislada* conforman la planta física bajo regulación. El foco calefactor transforma la potencia eléctrica modulada por el dimmer en energía térmica por efecto Joule dentro del recinto cerrado, donde la inercia térmica y la transferencia de calor hacia el exterior definen la dinámica temporal que el algoritmo PID debe estabilizar. Al tratarse de una carga resistiva aislada gobernada a través del dimmer, su operación es compatible con cualquier entorno de programación embebida como C, C++, MicroPython o Python.

Las *protoboards, cables jumper Dupont, caimanes de interconexión y fuente externa regulada de 5 V* garantizan el soporte estructural del circuito, la integridad de las señales digitales y el suministro de corriente independiente para el servomotor con referencia de tierra común hacia el ESP32.

Desarrollo de la Práctica

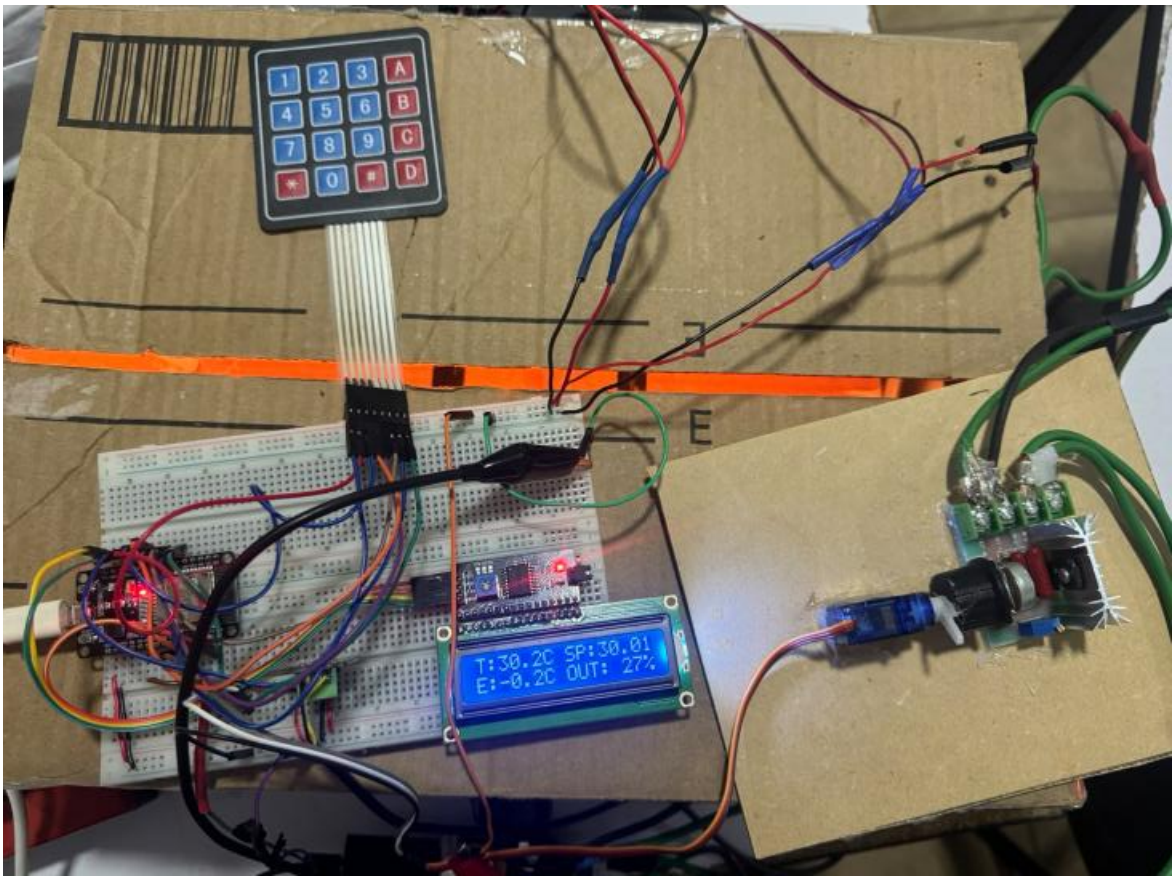
La implementación del controlador PID se llevó a cabo evolucionando la arquitectura electromecánica de la cámara térmica utilizada en la etapa proporcional previa, conservando el aislamiento físico entre la etapa de potencia y el circuito de control. En la cubierta superior del recinto se mantuvo fijado el conjunto formado por el dimmer regulador de corriente alterna y el servomotor SG90, alineando coaxialmente sus ejes y asegurando el acople mecánico sobre la perilla reguladora. Antes de iniciar las pruebas en lazo cerrado, se refinó la calibración experimental de la zona muerta del dimmer, determinando que el umbral mínimo de conducción comenzaba exactamente a los 80 grados y la potencia máxima se alcanzaba a los 180 grados, por lo que dichos límites angulares se definieron como ANGULO_MIN y ANGULO_MAX dentro del firmware.

En la protoboard montada sobre la cámara térmica se organizó la interconexión electrónica del microcontrolador ESP32 DevKit V1. El bus I2C de la pantalla LCD de 16×2 se conectó a los pines GPIO 21 para SDA y GPIO 22 para SCL bajo la dirección 0x27; la línea de datos del sensor DS18B20 se enlazó al pin GPIO 4 con su respectiva resistencia pull-up de 4.7 kΩ hacia 3.3 V; las cuatro filas y cuatro columnas del teclado matricial 4×4 se conectaron a los pines GPIO 13, 12, 14, 27, 26, 25, 33 y 32; y la línea de señal PWM del servomotor SG90 se conectó al pin GPIO 18, alimentando el actuador desde una fuente externa regulada de 5 V con tierra común hacia el ESP32.



El desarrollo del software se ejecutó en Visual Studio Code mediante PlatformIO bajo el framework de Arduino en C++, integrando las librerías OneWire, DallasTemperature, LiquidCrystal_I2C, Keypad y ESP32Servo. Para implementar un controlador PID discreto profesional y estable con un periodo de muestreo fijo Ts de 1.0 segundo, se programó la función calcularPID incorporando tres refinamientos fundamentales de control digital: en primer lugar, el término derivativo se calculó directamente sobre la variación de la temperatura medida en lugar del error para evitar saltos bruscos o *derivative kick* cada vez que el usuario modifica el setpoint; en segundo lugar, se implementó una técnica de *anti-windup clamping* que detiene la acumulación del término integral cuando la salida total alcanza los límites de saturación del 0 % o del 100 %, evitando sobreimpulsos prolongados; y en tercer lugar, se inicializó T_medida_anterior con la lectura térmica del arranque en setup para prevenir picos derivativos en el primer instante de ejecución.

Asimismo, se programó una máquina de estados gobernada por el teclado matricial que permite modificar con las teclas A, B, C y D los valores de T_ref, Kp, Ki y Kd respectivamente, introduciendo cifras decimales con * y confirmando con #. Paralelamente, la rutina de visualización alterna automáticamente cada cuatro segundos entre la pantalla de variables de proceso y la pantalla de diagnóstico de ganancias PID y posición angular del servomotor, enviando también toda la traza de datos por el puerto serie a 115200 baudios para su análisis gráfico.



Vista de diagnóstico en la pantalla LCD mostrando las ganancias P en 10.0, I en 0.5 y D en 1.0, junto con la posición angular del servomotor en 111 grados accionando la perilla del dimmer

```
#include <Arduino.h>
#include <Wire.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <LiquidCrystal_I2C.h>
#include <Keypad.h>
```

```
#include <ESP32Servo.h>

// ==========================================
// 1. PINES Y CALIBRACIÓN DE ZONA MUERTA
// ==========================================
#define DS18B20_PIN 4
#define SERVO_PIN   18
#define LCD_SDA     21
#define LCD_SCL     22
#define LCD_ADDR    0x27

// Calibración mecánica del Dimmer (Zona Muerta sintonizada
previamente)
const int ANGULO_MIN = 80;  // 0%  de potencia PID -> 80°
(Umbral inicial)
const int ANGULO_MAX = 180; // 100% de potencia PID -> 180°
(Máxima potencia)

// ==========================================
// 2. CONFIGURACIÓN TECLADO MATRICIAL 4x4
// ==========================================
const byte FILAS = 4;
const byte COLUMNAS = 4;

char teclas[FILAS][COLUMNAS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};

byte pinesFilas[FILAS]       = {13, 12, 14, 27};
byte pinesColumnas[COLUMNAS] = {26, 25, 33, 32};

Keypad teclado = Keypad(makeKeymap(teclas), pinesFilas,
pinesColumnas, FILAS, COLUMNAS);

// ==========================================
// 3. OBJETOS Y VARIABLES PID
// ==========================================
OneWire oneWire(DS18B20_PIN);
DallasTemperature sensors(&oneWire);
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);
Servo dimmerServo;

// Variables de proceso y referencia
float T_medida = 0.0;
float T_medida_anterior = 0.0;
float T_ref = 40.0;         // Setpoint inicial (°C)
float errorTemp = 0.0;

// Ganancias PID por defecto
float Kp = 10.0;            // Proporcional (%/°C)
float Ki = 0.2;             // Integral (%/(°C*s))
float Kd = 1.0;             // Derivativa (%*s/°C)

// Términos discretos del PID
float P_term = 0.0;
float I_term = 0.0;
```

```
float D_term = 0.0;
float accionPID = 0.0;      // Salida en % (0 a 100%)
int anguloServo = ANGULO_MIN;

// Periodo de Muestreo (Ts = 1 segundo)
const float Ts = 1.0;
unsigned long ultimaLecturaPID = 0;

// Máquina de estados para edición
enum ModoSistema { MODO_NORMAL, EDIT_SP, EDIT_KP, EDIT_KI,
EDIT_KD };
ModoSistema modoActual = MODO_NORMAL;

String bufferTeclado = "";
bool vistaAlternativaLCD = false;
unsigned long ultimoCambioPantalla = 0;

// ==========================================
// 4. ALGORITMO PID DISCRETO CON ANTI-WINDUP
// ==========================================
void calcularPID() {
  errorTemp = T_ref - T_medida;

  // 1. Término Proporcional
  P_term = Kp * errorTemp;

  // 2. Término Derivativo basado en la Medición (Evita el
"Derivative Kick")
  float dT = (T_medida - T_medida_anterior) / Ts;
  D_term = -Kd * dT;

  // 3. Acumulación Integral tentativa
  float I_tentativo = I_term + (Ki * errorTemp * Ts);

  // 4. Salida sin saturar
  float u_unclamped = P_term + I_tentativo + D_term;

  // 5. Anti-Windup Clamping (Detiene la acumulación integral al
saturar)
  if (u_unclamped >= 0.0 && u_unclamped <= 100.0) {
    I_term = I_tentativo; // Solo actualiza si está en el rango
lineal
  } else {
    I_term = constrain(I_term, -50.0, 50.0); // Limita el
acumulado
  }

  // 6. Saturación de la salida (0% - 100%)
  accionPID = constrain(u_unclamped, 0.0, 100.0);

  // 7. Conversión a ángulo del servomotor
  float anguloFlotante = ANGULO_MIN + (accionPID / 100.0) *
(ANGULO_MAX - ANGULO_MIN);
  anguloServo = (int)constrain(anguloFlotante,
(float)ANGULO_MIN, (float)ANGULO_MAX);

  // Actualizar actuador físico
  dimmerServo.write(anguloServo);
```

```
  // Guardar medición previa para el cálculo derivativo
  T_medida_anterior = T_medida;
}

// ========================================
// 5. INTERFAZ DE USUARIO LCD
// ========================================
void actualizarLCD() {
  char linea0[17];
  char linea1[17];

  switch (modoActual) {
    case EDIT_SP:
      snprintf(linea0, sizeof(linea0), "Set T_ref (C):  ");
      snprintf(linea1, sizeof(linea1), "> %-14s",
bufferTeclado.c_str());
      break;
    case EDIT_KP:
      snprintf(linea0, sizeof(linea0), "Set Kp:         ");
      snprintf(linea1, sizeof(linea1), "> %-14s",
bufferTeclado.c_str());
      break;
    case EDIT_KI:
      snprintf(linea0, sizeof(linea0), "Set Ki:         ");
      snprintf(linea1, sizeof(linea1), "> %-14s",
bufferTeclado.c_str());
      break;
    case EDIT_KD:
      snprintf(linea0, sizeof(linea0), "Set Kd:         ");
      snprintf(linea1, sizeof(linea1), "> %-14s",
bufferTeclado.c_str());
      break;
    case MODO_NORMAL:
    default:
      if (!vistaAlternativaLCD) {
        // Vista 1: Control (T_medida, Setpoint, Error, OUT%)
        snprintf(linea0, sizeof(linea0), "T:%4.1fC SP:%4.1f",
T_medida, T_ref);
        snprintf(linea1, sizeof(linea1), "E:%4.1fC OUT:%3d%%",
errorTemp, (int)accionPID);
      } else {
        // Vista 2: Diagnóstico (Ganancias PID y Posición Servo)
        snprintf(linea0, sizeof(linea0), "P:%.1f I:%.1f D:%.1f",
Kp, Ki, Kd);
        snprintf(linea1, sizeof(linea1), "SERVO: %3d DEG  ",
anguloServo);
      }
      break;
  }

  lcd.setCursor(0, 0);
  lcd.print(linea0);
  lcd.setCursor(0, 1);
  lcd.print(linea1);
}

void procesarTeclado(char tecla) {
  if (tecla == 'A') {
    modoActual = EDIT_SP;
```

```
    bufferTeclado = "";
  } else if (tecla == 'B') {
    modoActual = EDIT_KP;
    bufferTeclado = "";
  } else if (tecla == 'C') {
    modoActual = EDIT_KI;
    bufferTeclado = "";
  } else if (tecla == 'D') {
    modoActual = EDIT_KD;
    bufferTeclado = "";
  } else if (tecla == '#') { // Confirmar e ingresar el
parámetro
    if (bufferTeclado.length() > 0) {
      float val = bufferTeclado.toFloat();
      if (modoActual == EDIT_SP && val >= 0.0 && val <= 100.0)
T_ref = val;
      else if (modoActual == EDIT_KP && val >= 0.0 && val <=
100.0) Kp = val;
      else if (modoActual == EDIT_KI && val >= 0.0 && val <=
50.0)  Ki = val;
      else if (modoActual == EDIT_KD && val >= 0.0 && val <=
50.0)  Kd = val;
    }
    modoActual = MODO_NORMAL;
    bufferTeclado = "";
  } else if (tecla == '*') {
    if (modoActual != MODO_NORMAL) {
      // Punto decimal
      if (bufferTeclado.indexOf('.') == -1 &&
bufferTeclado.length() < 6) {
        bufferTeclado += '.';
      }
    } else {
      // Alternar manualmente la vista en modo normal
      vistaAlternativaLCD = !vistaAlternativaLCD;
    }
  } else if (tecla >= '0' && tecla <= '9') {
    if (modoActual != MODO_NORMAL && bufferTeclado.length() < 6)
{
      bufferTeclado += tecla;
    }
  }
}

// ==========================================
// 6. SETUP Y LOOP
// ==========================================
void setup() {
  Serial.begin(115200);

  // Inicialización Servo
  ESP32PWM::allocateTimer(0);
  dimmerServo.setPeriodHertz(50);
  dimmerServo.attach(SERVO_PIN, 500, 2400);
  dimmerServo.write(ANGULO_MIN);

  // Inicialización LCD I2C
  Wire.begin(LCD_SDA, LCD_SCL);
  lcd.init();
```

```
  lcd.backlight();

  // Inicialización DS18B20
  sensors.begin();
  sensors.setResolution(10); // Resolución a 10 bits
  sensors.setWaitForConversion(false);
  sensors.requestTemperatures();

  // Guardar medición inicial para evitar saltos derivativos en
t=0
  float tempInit = sensors.getTempCByIndex(0);
  if (tempInit > -55.0 && tempInit < 125.0) {
    T_medida = tempInit;
    T_medida_anterior = tempInit;
  }

  lcd.setCursor(0, 0);
  lcd.print(" Control PID    ");
  lcd.setCursor(0, 1);
  lcd.print(" Temperatura    ");
  delay(1500);
  lcd.clear();
}

void loop() {
  unsigned long ahora = millis();

  // 1. Lectura del Teclado Matricial
  char tecla = teclado.getKey();
  if (tecla != NO_KEY) {
    procesarTeclado(tecla);
    actualizarLCD();
  }

  // 2. Alternancia automática de vistas LCD cada 4 segundos
(solo en MODO_NORMAL)
  if (modoActual == MODO_NORMAL && (ahora - ultimoCambioPantalla
>= 4000)) {
    vistaAlternativaLCD = !vistaAlternativaLCD;
    ultimoCambioPantalla = ahora;
    actualizarLCD();
  }

  // 3. Lazo de Control PID (Periodo de Muestreo Ts = 1.0 s)
  if (ahora - ultimaLecturaPID >= (unsigned long)(Ts * 1000)) {
    ultimaLecturaPID = ahora;

    float tempRead = sensors.getTempCByIndex(0);
    if (tempRead > -55.0 && tempRead < 125.0) {
      T_medida = tempRead;
    }
    sensors.requestTemperatures(); // Solicitar siguiente
conversión para el próximo ciclo

    // Ejecutar algoritmo de control PID
    calcularPID();
    actualizarLCD();

    // Monitor Serie (Ideal para graficar en Serial Plotter)
```

```
Serial.printf("T_medida:%.2f,T_ref:%.2f,Error:%.2f,P:%.2f,I:%.2f
,D:%.2f,OUT:%.1f,Servo:%d\n",
                  T_medida, T_ref, errorTemp, P_term, I_term,
D_term, accionPID, anguloServo);
  }

                                  }
```

Diagrama Repositorio en Github

Resultados

Las pruebas experimentales realizadas sobre la cámara térmica confirmaron un desempeño sobresaliente del controlador PID discreto frente a los esquemas ON-OFF y proporcional evaluados anteriormente. Durante la sintonización en tiempo real mediante el teclado matricial 4×4, se establecieron ganancias iniciales de Kp igual a 10.0, Ki igual a 0.2 y Kd igual a 1.0, ajustando posteriormente la ganancia integral a 0.5 para acelerar la corrección del error residual y verificar la respuesta dinámica del sistema ante distintos puntos de consigna.

El hallazgo experimental más relevante quedó evidenciado al alcanzar el régimen estacionario alrededor de una temperatura de referencia de 30.0 °C. A diferencia del controlador puramente proporcional donde un error nulo o ligeramente negativo provocaba que la salida cayera al 0 % dejando enfriar el horno, la acción integral del controlador PID acumuló el histórico del error térmico y sostuvo una acción de control estable del 27 %, posicionando el servomotor alrededor de 111 grados para suministrar exactamente la potencia mínima continua necesaria que compensa las pérdidas de calor hacia el ambiente. De este modo, la temperatura medida se estabilizó en 30.2 °C con un error mínimo de -0.2 °C, eliminando tanto las oscilaciones bruscas del control todo-nada como el error permanente del control proporcional.

De igual manera, la protección anti-windup y el cálculo derivativo sobre la medición evitaron movimientos erráticos o saturaciones prolongadas del servomotor SG90 ante cambios escalón en la temperatura de referencia, garantizando transiciones suaves sobre la perilla del dimmer y una lectura clara en ambas vistas de la pantalla LCD.

Conclusiones

La implementación de este sistema permitió comprobar de manera integral y experimental el funcionamiento de un lazo cerrado de control PID discreto aplicado a un proceso térmico real. Se demostró que la incorporación de la acción integral resulta determinante para eliminar el error en estado estable, permitiendo que el sistema mantenga un porcentaje de potencia sostenido sobre el dimmer para equilibrar exactamente las pérdidas térmicas del recinto una vez alcanzada la temperatura de referencia, mientras que la acción derivativa aporta amortiguamiento frente a variaciones rápidas de temperatura.

Asimismo, se validó la importancia de aplicar técnicas de ingeniería de control digital como el anti-windup, la derivación sobre la variable medida, la calibración de zona muerta de actuadores mecánicos y el muestreo periódico no bloqueante en el microcontrolador ESP32. Con la culminación exitosa de esta práctica se consolidaron los conocimientos adquiridos a lo largo del curso sobre instrumentación con sensores digitales, interfaces hombre-máquina, electrónica de potencia aislada, accionamiento electromecánico y diseño de controladores automáticos embebidos.

Referencias

Analog Devices. *DS18B20: Programmable Resolution 1-Wire Digital Thermometer Datasheet*. Analog Devices, Inc., 2019.

Åström, K. J., & Hägglund, T. *Control PID avanzado*. Pearson Educación, 2009.

Bolton, W. *Ingeniería de control*. Alfaomega Grupo Editor, 2013.

Espressif Systems. *ESP32 Series Datasheet*. Espressif Systems Co., Ltd., 2023.

Ogata, K. *Ingeniería de control moderna*. Pearson Educación, 2010.

PlatformIO. *PlatformIO Documentation: Espressif 32 Platform and Arduino Framework*. PlatformIO Labs, 2024.

Tower Pro. *SG90 9g Micro Servo Technical Specification Sheet*. Tower Pro Pte Ltd, 2018.